An

Industrial Training Report

On

Data Analytics

Submitted partial fulfilment for the award of degree of

Bachelor of Technology

In

Computer Science and Engineering

Submitted to:

Dr. Pradeep Jha

HOD-CSE/IT/AI&DS/IOT Dept.

Submitted By:

Harsh Kumar Sharma

24EGJCS086

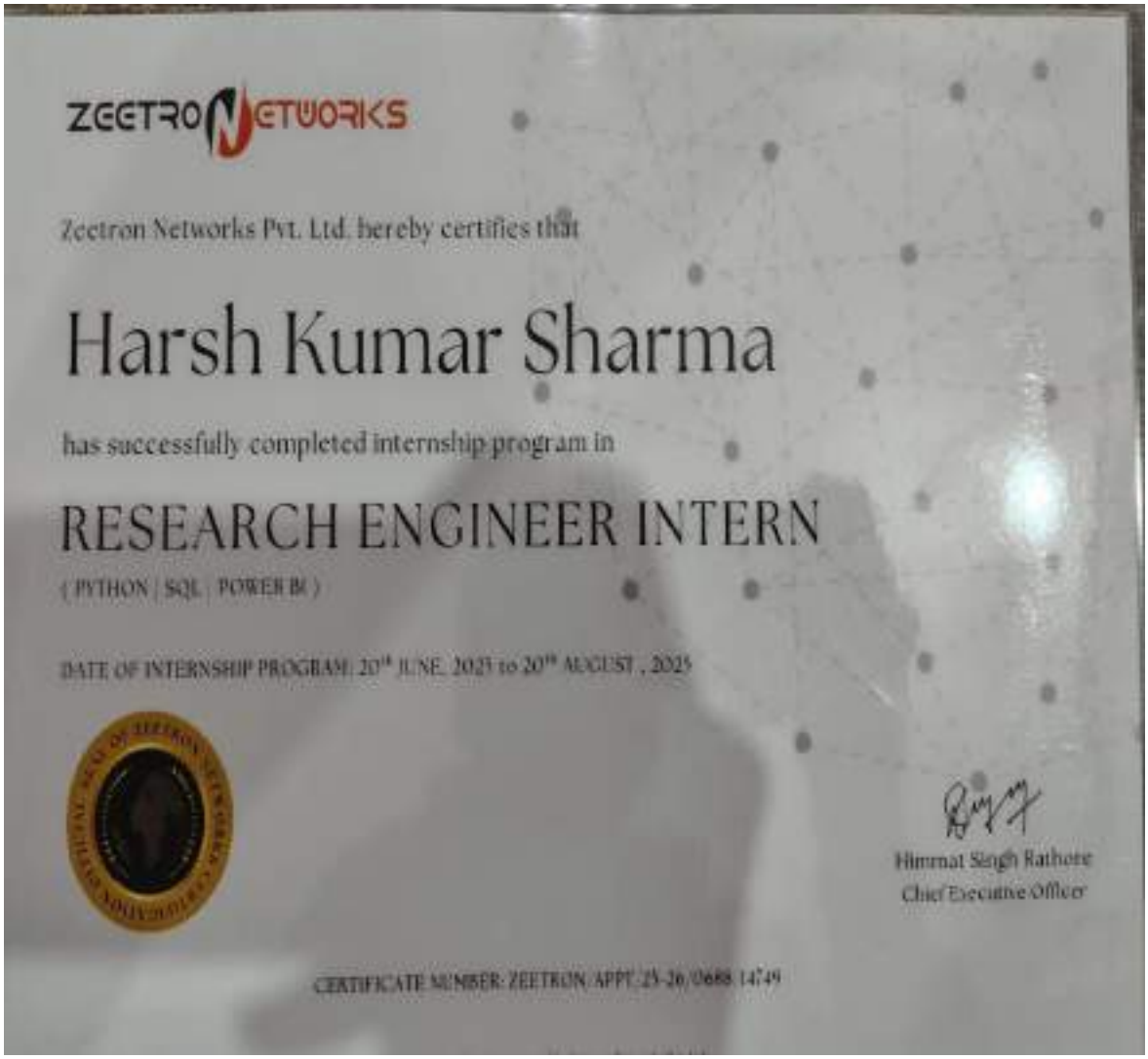
Department of Computer Science & Engineering

Global Institute of Technology

Jaipur (Rajasthan)-302022

Session: 2025-26

Certificate



Acknowledgement

I express my sincere gratitude to **Mr. AshaRam Gurjar**, Assistant Professor, Department of Computer Science and Engineering, Global Institute of Technology, Jaipur, for providing me the opportunity to undergo Industrial Training in the domain of **Data Analytics.** His valuable guidance, continuous support, and insightful feedback played a significant role in enhancing my understanding.

I extend my heartfelt thanks to the entire team of **GIT, Jaipur** for their support and encouragement throughout the training period. Their expertise, real-time examples, and project-based mentoring helped me gain practical exposure to modern technologies such as NumPy, Pandas, matplotlib, and seaborn.

Place: GIT , Jaipur,

Harsh Kumar Sharma,

24EGJCS086,

B.Tech III Semester, II Year, CSE

Abstract

This report presents a comprehensive data analytics project that integrates **Python programming**, **SQL querying**, and **Power BI** visualization to extract meaningful insights from complex datasets. The workflow begins with data acquisition and preprocessing, where Python libraries such as **Pandas** and **NumPy** are utilized to clean, structure, and transform raw data into an analysis-ready format. Advanced exploratory data analysis techniques are applied to identify trends, correlations, outliers, and potential data quality issues. **SQL** plays a critical role in efficiently accessing and manipulating data stored within relational databases, enabling precise filtering, aggregation, and joining of tables to support deeper analytical tasks.

Building on this foundation, predictive and statistical models are developed using Python's machine-learning ecosystem, including scikit-learn and related libraries. These models help uncover hidden patterns and forecast outcomes with improved accuracy compared to traditional reporting methods. Insights generated from the analysis are then translated into clear, interactive visual representations using Power BI. Dashboards and reports highlight key performance indicators, emerging trends, and actionable insights that can be easily interpreted by both technical and non-technical stakeholders.

The findings of this project demonstrate the effectiveness of combining programming, database management, and business intelligence tools to support data-driven decision-making. Limitations related to data quality, model generalization, and scalability are discussed, along with recommendations for improving analytical robustness and expanding future capabilities. Overall, this work showcases how modern data analytics techniques can transform raw information into valuable knowledge that supports strategic planning and operational efficiency.

Table of Contents

Chapters

List of Figures

List of Tables

Chapter 1

Introduction of data analysis and visualization in today's world

In an era where data is often referred to as the "new oil," the ability to effectively analyze and visualize data has become crucial across various sectors. From business to healthcare, education to social sciences, the insights gleaned from data analysis drive critical decision-making processes and enhance overall efficiency.

1.1 What is Data Analytics:

Data Analytics is a systematic process of examining raw data to extract meaningful insights, patterns, and conclusions. In today's digital world, huge amounts of data are generated from various sources such as websites, mobile apps, transactions, sensors, and business operations. Data analytics helps convert this unstructured or semi-structured data into structured, understandable information.

The process typically involves several stages, including data collection, data cleaning, data transformation, modeling, and interpretation. Cleaning ensures that incorrect or missing values are fixed so that the results are accurate.

1.2 Importance of Data Analysis:

Data analysis involves systematically applying statistical and logical techniques to describe and illustrate, condense and recap, and evaluate data. The primary goals of data analysis include:

- Identifying Trends and Patterns: Businesses utilize data analysis to uncover patterns that can inform strategy. For example, retail companies analyze purchasing habits to forecast inventory needs.

- Improving Decision-Making: With robust data analysis, organizations can base decisions on empirical evidence rather than intuition, thereby minimizing risks and maximizing opportunities.

1.3 What Data Analytics Actually Does:

Data Analytics takes **raw data** (numbers, records, images, clicks, etc.) and turns it into **useful information** that helps people and companies make better decisions.

1. Collects Data :

Data Analytics begins by gathering data from different sources such as websites, apps, surveys, machines, and transactions.
This raw data can be structured, semi-structured, or unstructured.
The purpose of collecting data is to understand user behavior or business performance.
Companies often store this data in databases or cloud systems.
It forms the base for all further analysis.

2. Cleans and Organizes the Data :

Raw data usually contains errors, missing values, duplicates, or irrelevant entries.
Data cleaning removes these issues to ensure accuracy and reliability.
After cleaning, data is arranged properly for easy analysis.

Organized data helps prevent confusion and improves decision-making quality. This step is essential for getting correct results.

3. Finds Patterns and Trends :

Analytics studies data to discover hidden patterns and trends over time. It tells what is increasing, decreasing, or changing in the business. Examples include best-selling products, peak activity hours, or customer preferences. These patterns help companies understand their overall performance. It provides a clear picture of what is happening in the data.

4. Answers Important Questions :

Analytics helps answer key business questions using data. It explains what happened, why it happened, and what may happen next. This reduces guesswork and supports fact-based decisions. Managers use these insights to understand problems or opportunities. Better clarity leads to smarter strategies.

5. Makes Predictions:

Data Analytics uses past data to forecast future outcomes. It predicts things like future sales, demand, customer behavior, or trends. These predictions help companies prepare in advance. This improves planning in areas like inventory, staffing, and marketing. It gives businesses a competitive advantage.

6. Helps Companies Take Better Actions:

After analysis, companies receive suggestions on what actions to take. These actions help reduce costs, improve performance, and increase profits. Analytics also supports smarter marketing and better resource management. Businesses use these insights to avoid risks and identify opportunities. This leads to more effective decision-making.

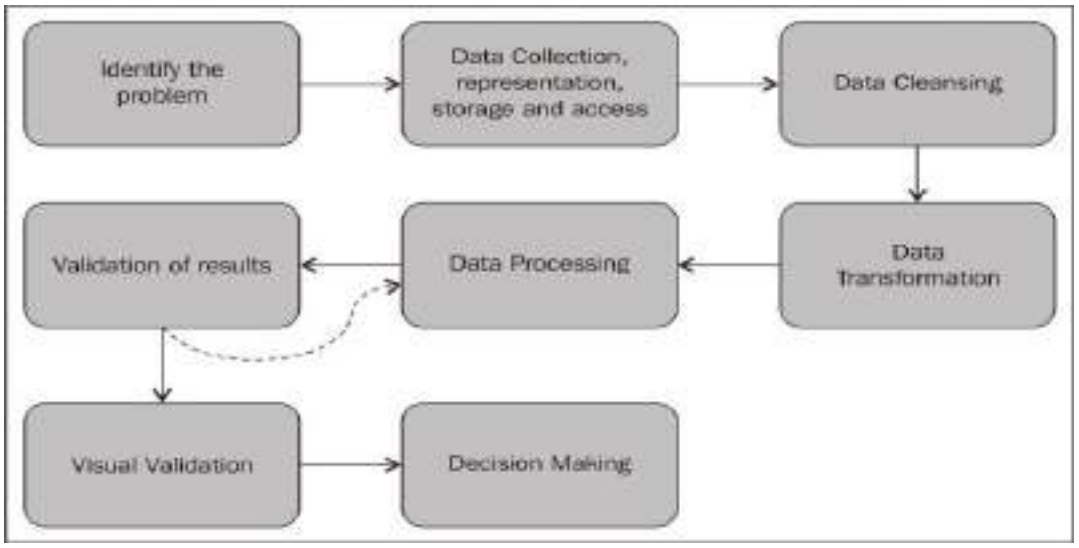


Figure 01:Workflow

1.4 The Role of Data Visualization:

Data visualization plays a pivotal role in making the results of data analysis comprehensible and accessible. It is the graphical representation of information and data, enabling stakeholders to:

- Simplify Complex Data: Visualization simplifies complex data sets, presenting them in a form that is easy to understand. This reduces cognitive overload and allows for quicker insights.

- Highlight Key Findings: Interactive charts and graphs can illuminate significant trends or anomalies in data, helping users grasp essential insights at a glance.
- Facilitate Data Storytelling: Visualization not only conveys numbers but also tells a story. A well-designed visualization can guide viewers through the data, offering context and interpretation that supports the narrative.

1.5 Key Technologies Used in Data Analysis and Visualization:

In the context of data analysis and visualization, several key technologies serve as foundational tools that help transform raw data into actionable insights. Below are the primary technologies commonly utilized, along with a brief overview of each

1. Programming Languages:

- **Python:** A versatile language widely used for data analysis due to its rich ecosystem of libraries such as Pandas, NumPy, and Matplotlib. Python's ease of use makes it accessible for data scientists, analysts, and developers.

2. Database Management Systems (DBMS):

- **MySQL:** An open-source relational database that efficiently manages structured data through SQL queries, making it one of the most widely-used databases for web applications and data storage.

3. Data Visualization Tools:

- **Power BI:** A business analytics tool by Microsoft that allows users to create interactive reports and dashboards. Its integration with various data sources and its user-friendly interface make it popular among business users.

CHAPTER 2

INTRODUCTION TO PYTHON PROGRAMMING

Python is one of the most widely used programming languages in the world due to its simplicity, readability, and extensive support for data-related operations. In the field of Data Analytics, Python has become the preferred language because it combines powerful libraries, strong community support, and flexibility for handling structured and unstructured data.

This chapter explores the basics of Python, its architecture, data types, operators, functions, modules, and error handling. These foundational concepts were covered during the internship and helped build the analytical skills required for the subsequent chapters.

2.1 Overview of Python

Python is a high-level, interpreted, object-oriented programming language. It was created by Guido van Rossum and first released in 1991. Python emphasizes code readability, allowing developers to write fewer lines of code compared to other programming languages like Java or C++.

Key Features of Python

Easy to Read & Write – Uses clean syntax similar to English.

Interpreted Language – Executes line by line, making debugging easier.

Extensive Libraries – Supports data science, AI, web development, automation, etc.

Object-Oriented – Supports encapsulation, inheritance, and polymorphism.

Platform Independent – Works on Windows, macOS, Linux.

Community Support – One of the largest developer communities in the world.

2.2 Python Architecture

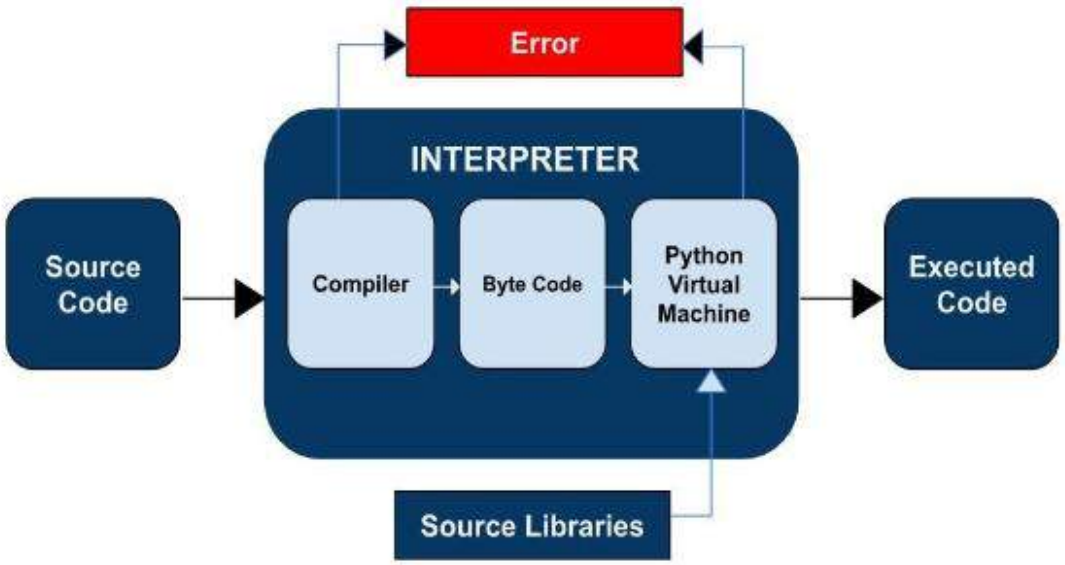


Figure 2.1: Python Execution Architecture

Python code goes through these steps:

Source Code (.py file)

Python Interpreter converts code into:

Byte Code

Python Virtual Machine (PVM) executes the byte code

Output

This architecture makes Python easy to run across multiple operating systems.

2.3 Variables and Data Types

Variables in Python are created automatically when a value is assigned. Python is dynamically typed, meaning the type of variable is inferred at runtime.

Example:

```
x = 10          # int
name = "John"   # string
```

```
price = 75.5    # float
```

Table 2.1: Python Data Types

Category	Data Type	Example
Numeric	int,float,complex	10,3.14,2+3j
Sequence	list,tuple,range	[1,2,3],(a,b)
Text	str	“Hello”
Mapping	dict	{“Id”,1}
Boolean	bool	True/False
Set	Set,frozenset	{1,2,3}

These data types form the backbone for processing datasets in Pandas and NumPy.

2.4 Operators in Python

Python supports various operators:

Arithmetic Operators: +, -, *, /, %, //, **

Relational Operators: >, <, >=, <=, ==

Logical Operators: and, or, not

Assignment Operators: =, +=, -=

Membership Operators: in, not in

Identity Operators: is, is not

These operators are important for writing conditional statements and loops.

2.5 Conditional Statements

Python supports decision-making using:

Example:

```
if score >= 90:
    grade = "A"
elif score >= 75:
    grade = "B"
else:
    grade = "C"
```

Control flow statements help implement logical checks during data processing.

2.6 Looping Statements

For Loop

Used to iterate through sequences such as lists or strings.

```
for i in range(5):
    print(i)
```

While Loop

Used when the number of iterations is unknown.

```
while x < 10:
    x += 1
```

Loops are heavily used in data cleaning and transformation.

2.7 Functions in Python

Functions allow code reusability and modular programming.

Defining a function:

```
def add(a, b):
    return a + b
```

Calling a function:

```
result = add(5, 3)
```

Types of functions:

- **User-defined functions**
- **Built-in functions (e.g., len(), sum())**
- **Lambda (anonymous) functions**

2.8 Modules and Packages

Python modules are files containing functions or variables.

Example:

```
import math

print(math.sqrt(16))
```

packages are collections of modules stored in a directory with _init_.py.

Popular packages:

- **numpy**
- **pandas**
- **matplotlib**

These form the foundation of data analysis.

2.9 Exception Handling

Exception handling ensures a program doesn't crash unexpectedly.

Example:

```
try:
    value = int(input("Enter number: "))
except ValueError:
    print("Invalid input!")
finally:
    print("Program executed.")
```

CHAPTER 3

PYTHON LIBRARIES FOR DATA ANALYTICS

Python's powerful ecosystem of libraries is one of the primary reasons it has become the leading language for Data Analytics. These libraries simplify tasks such as data cleaning, numerical computation, visualization, statistical modeling, and machine learning.

During the internship, the major focus was on understanding and applying four essential libraries:

- **NumPy**
- **Pandas**
- **Matplotlib**
- **Seaborn**

This chapter explores each library in detail with examples, use cases, and role in the data analytics workflow.

3.1 Introduction to Data Analytics Libraries

Data analytics often involves handling large datasets, performing mathematical operations, visualizing trends, and executing transformations. Python libraries help automate these processes efficiently.

Why Python Libraries are Important ?

Enhance productivity

Reduce boilerplate code

Handle complex data operations

Provide optimized performance

Enable visualization and statistical modeling

Together, these libraries form the core toolkit of every data analyst.

3.2 NumPy: Numerical Python

NumPy is the fundamental package for numerical computation in Python. It provides support for multi-dimensional arrays, mathematical functions, and linear algebra operations.

Key Features of NumPy

- **ndarray object (N-dimensional array)**
- **Vectorized operations (fast execution)**
- **Mathematical functions (sin, cos, log)**
- **Random number generation**
- **Broadcasting capability**

Example: Creating an Array

```
import numpy as np

arr = np.array([1, 2, 3, 4])
```

Use in Data Analytics:

NumPy is used for preprocessing, numerical simulation, and as the foundation for Pandas.

3.3 Pandas: Data Analysis Library

Pandas is the most widely used library in Data Analytics. It provides two powerful data structures:

Series (1D)

DataFrame (2D)

Pandas allows data cleaning, manipulation, filtering, merging, grouping, and reshaping.

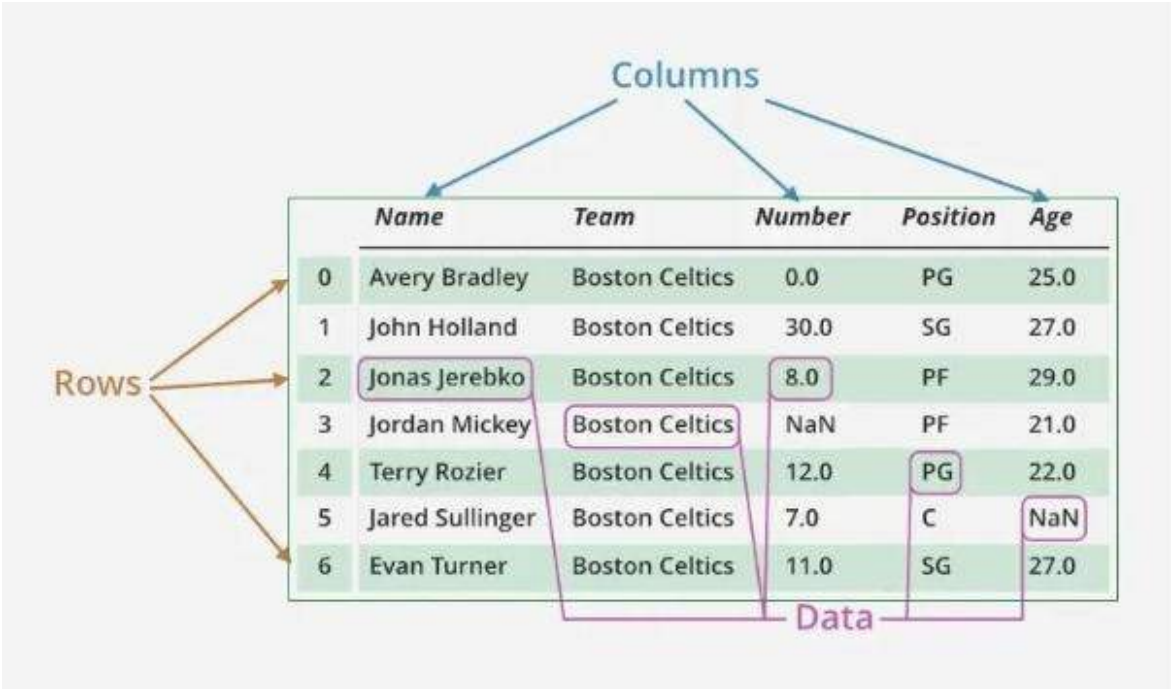


Figure 3.2: Pandas DataFrame Structure

Key Features of Pandas

- Load data from CSV, Excel, SQL
- Handle missing values
- Merge and join datasets
- GroupBy operations
- Data aggregation
- Date and time functionality

Example: Reading a Dataset

```
import pandas as pd

df = pd.read_csv("data.csv")
```

Example: Cleaning Data

```
df.dropna(inplace=True)

df['Age'] = df['Age'].fillna(df['Age'].mean())
```

Example: Grouping Data

```
df.groupby('Department')['Salary'].mean()
```

Use in Analytics:

Pandas is the backbone for data cleaning and EDA (Exploratory Data Analysis).

3.4 Matplotlib: Data Visualization Library

Matplotlib is a basic but powerful plotting library used to create high-quality charts and graphs.

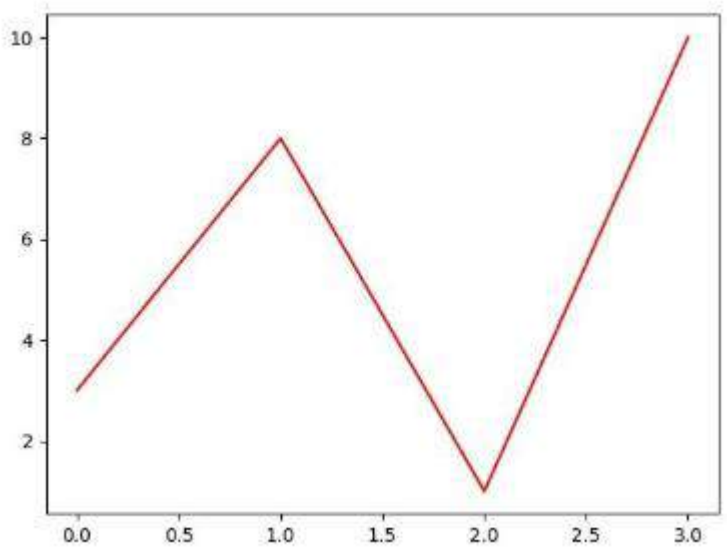


Figure 3.3: Example Matplotlib Line Plot

- Key Features of Matplotlib(line chart ,boxplot, scatter chart etc.)Example: Simple Plot

```
import matplotlib.pyplot as plt

plt.plot([1, 2, 3, 4], [2, 4, 6, 8])

plt.title("Simple Line Plot")

plt.xlabel("X-axis")

plt.ylabel("Y-axis")

plt.show()
```

CHAPTER 4

OBJECT-ORIENTED PROGRAMMING (OOP) IN PYTHON

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects," which bundle data and functions into a single unit. OOP makes software modular, reusable, scalable, and easier to maintain. Python is a strongly object-oriented language, meaning almost everything (variables, functions, modules) is treated as an object.

During the internship, understanding OOP helped in structuring Python programs efficiently and laid the foundation for real-world applications used in data analytics, automation, and model building.

4.1 Introduction to OOP

OOP solves problems using objects that interact with each other. Each object represents an entity having attributes and behaviour's.

- Easier debugging and maintenance
- Supports modular design

4.2 OOP Concepts: Class and Object

A class is a blueprint that defines the structure and behavior of objects.

An object is an instance of a class.

4.3 Diagram of OOP Structure

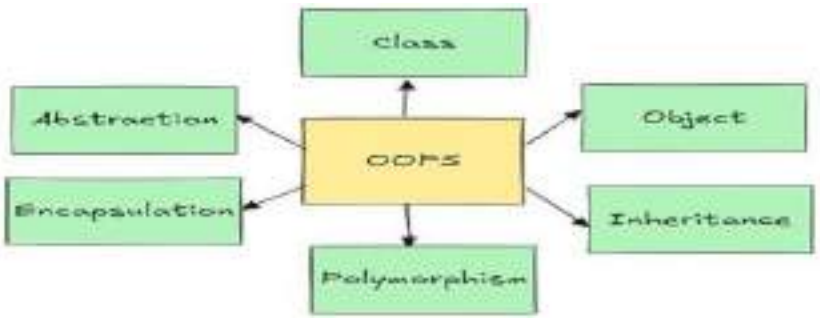


Figure 4.1: OOP Structure and Components

4.4 Encapsulation

Encapsulation bundles data and functions within a class and restricts direct access to internal variables.

Example:

```
class Student:
    def _init_(self):
        self.__marks = 90  # private variable
    def get_marks(self):
        return self.__marks
```

Here, __marks cannot be accessed directly from outside the class.

Benefit: Provides security and protects data.

4.5 Inheritance

Inheritance allows one class (child) to acquire attributes and methods of another class (parent).

Example:

```
class Animal:
    def speak(self):
        print("Animal speaks")
class Dog(Animal):
    pass
d = Dog()
d.speak()
```

Types of Inheritance in Python

- **Single**
- **Multiple**
- **Multilevel**
- **Hierarchical**
- **Hybrid**

4.6 Polymorphism

Polymorphism means “many forms.” Different classes can define methods with the same name but different behavior.

Example:

```
class Cat:
    def sound(self):
        return "Meow"
class Cow:
    def sound(self):
        return "Moo"
for animal in (Cat(), Cow()):
    print(animal.sound())
```

Here, the sound() method behaves differently depending on the object.

4.7 Abstraction

Abstraction hides unnecessary details and exposes only essential features.

Example using ABC module:

```
from abc import ABC, abstractmethod
class Vehicle(ABC):
    @abstractmethod
```

```
    def mileage(self):
pass
```

Benefit: Helps in designing cleaner, simpler, and more secure APIs.

4.8 Constructors in Python

Constructors initialize object variables automatically.

```
class Car:
    def _init_(self, brand, model):
        self.brand = brand
        self.model = model
```

init() is called automatically when the object is created.

4.9 OOP in Data Analytics

Although OOP is traditionally used in software development, it also plays an important role in Data Analytics:

Applications

Creating reusable data preprocessing pipelines

Defining model classes (e.g., in Scikit-Learn)

Automating ETL workflows

Example: Data Processor Class

```
class DataProcessor:
    def _init_(self, df):
        self.df = df
    def clean(self):
        self.df = self.df.dropna()
        return self.df
```

This demonstrates how OOP helps structure analytics tasks

CHAPTER 5

FILE HANDLING & OS MODULE IN PYTHON

File handling is one of the most essential components of data analytics because datasets are often stored in formats like CSV, Excel, JSON, or text files. Python provides powerful built-in functions for reading and writing files, while the OS module enables interaction with the operating system for tasks such as directory creation, file searching, automation, and system-level operations..

5.1 Introduction to File Handling

File handling refers to performing operations like:

1. **Reading**
2. **Writing**
3. **Appending**
4. **Deleting**

Different File Modes

Mode Description

"r" Read mode (default)

"w" Write mode (overwrites)

"a" Append mode

"b" Binary mode

"x" Create new file

Why File Handling Matters in Analytics

Loading large datasets

Exporting cleaned data

5.2 Reading Files in Python

Example: Reading a Text File

```
file = open("data.txt", "r")

content = file.read()

file.close()

Using with Statement (Recommended)

with open("data.txt", "r") as f:

    content = f.read()
```

Benefits of with:

Automatically closes file

Prevents memory leaks

Reading Line by Line

with open("data.txt") as f:

  for line in f:

    print(line)

5.3 Writing and Appending Data

Writing to a File

```
with open("output.txt", "w") as f:

    f.write("Hello World!")
```

Appending Data

```
with open("output.txt", "a") as f:

    f.write("¥nNew Line Added")
```

Writing Lists to a File

```
lines = ["A¥n", "B¥n", "C¥n"]

with open("letters.txt", "w") as f:
```

```
    f.writelines(lines)
```

5.4 Handling Excel Files

Python supports Excel operations through openpyxl and Pandas.

Example

import pandas as pd

df = pd.read_excel("data.xlsx")

Excel files are commonly used in business analytics, HR data, sales reports, etc.

5.5 The OS Module in Python

The os module allows Python to interact with the operating system.

Importing OS

import os

5.6 Directory and File Operations

Get Current Working Directory

```
os.getcwd()
```

List Files in a Directory

```
os.listdir()
```

Create a New Folder

```
os.mkdir("NewFolder")
```

Remove a File

```
os.remove("oldfile.txt")
```

Check if a Path Exists

```
os.path.exists("data.csv")
```

5.7 Automation Using OS Module

Automation is extremely useful in data analytics for tasks like:

Automatically importing monthly reports

Renaming large numbers of files

Creating dated folders for output

Scheduling scripts

Example: Automating File Movement

```
import os

import shutil

for file in os.listdir():

    if file.endswith(".csv"):

        shutil.move(file, "CSV_Files/")
```

CHAPTER 6

SQL & MySQL FOR DATA MANAGEMENT

SQL (Structured Query Language) is the most widely used language for managing and manipulating data stored in relational databases. In data analytics, structured data is often stored in tables, making SQL an essential skill for data querying, filtering, transformation, and aggregation.

MySQL, an open-source relational database management system (RDBMS), supports SQL and is used extensively in enterprise analytics workflows.

.

6.1 Introduction to Databases

A database is an organized collection of data stored electronically.

A DBMS (Database Management System) enables efficient storage, retrieval, and manipulation of data.

6.2 What is SQL?

SQL is a standardized language used to:

- **Create tables**
- **Insert and modify data**
- **Retrieve data using queries**
- **Join tables**
- **Filter and aggregate data**

6.3 MySQL Overview

MySQL is a relational database that stores data in structured tables using rows and columns. It is known for:

- High performance

- Reliability
- Scalability
- Cross-platform support

It is widely used in analytics systems, web applications, and business dashboards.

6.4 Relational Model Diagram

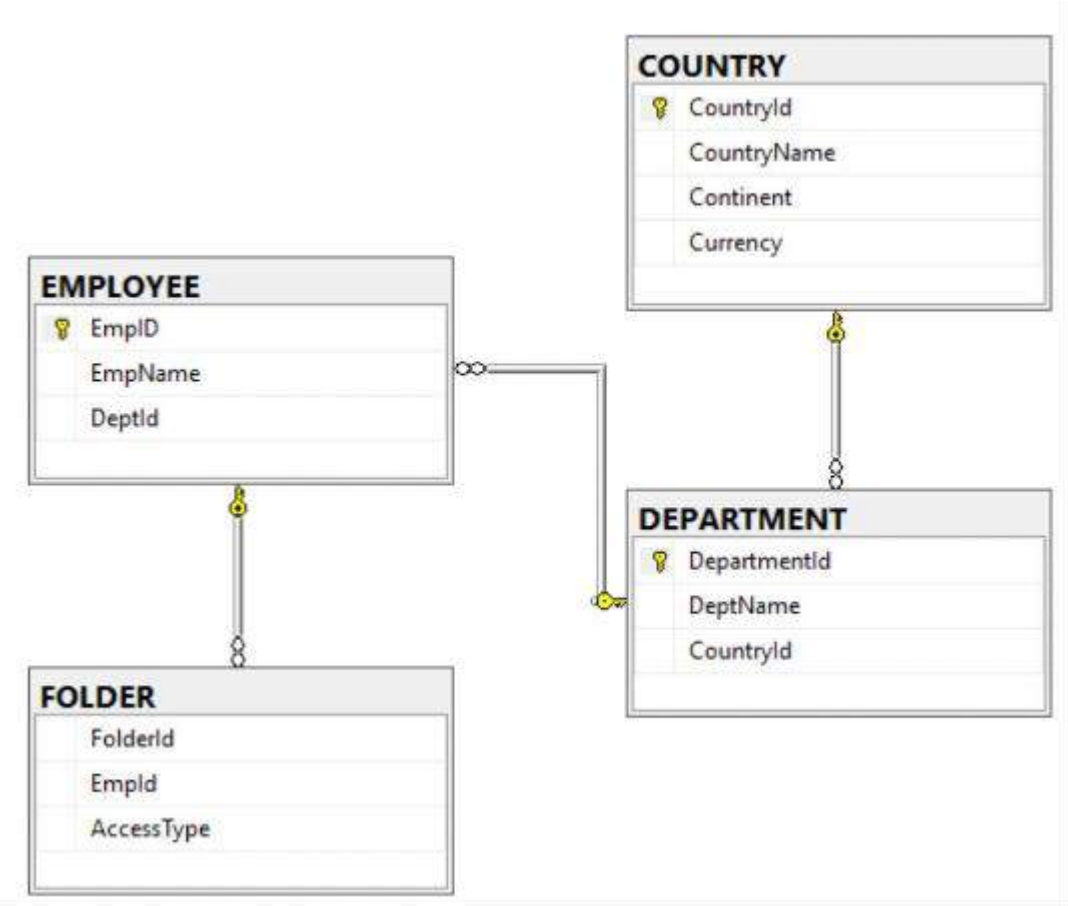


Figure 6.1: Relational Database Model (Tables linked through Primary and Foreign Keys)

The figure shows how data is organized into tables that relate to each other using keys.

6.5 SQL Components

SQL consists of several categories of commands.

Category Commands Purpose

DDL CREATE, ALTER, DROP Define database structure

DML SELECT, INSERT, UPDATE, DELETE Manage data inside tables

DCL GRANT, REVOKE Control access

TCL COMMIT, ROLLBACK Manage transactions

6.6 Creating a Database and Table in MySQL

```
Create Database

CREATE DATABASE analytics_db;

Use Database

USE analytics_db;

Create Table

CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    salary FLOAT
);

;
```

6.7 Constraints in SQL

Constraints enforce rules on data.

- PRIMARY KEY – Unique record identifier
- FOREIGN KEY – Links tables
- UNIQUE – Prevents duplicate values
- NOT NULL – Ensures data presence
- CHECK – Restricts value range
- ALTER TABLE employees
- ADD CONSTRAINT age_check CHECK (age > 18);

6.8 Aggregation Functions

Used for summarizing data in analytics.

Examples:

SELECT COUNT(*) FROM employees;

SELECT AVG(salary) FROM employees;

SELECT MAX(salary), MIN(salary) FROM employees;

Aggregation is frequently used in dashboards and business reports.

6.9 SQL Usage in Real-World Analytics

Common Applications

- Extracting sales reports
- Filtering customer segments
- Identifying top-performing regions
- Cleaning and combining multiple datasets
- Feeding dashboards like Power BI

CHAPTER 7

INTEGRATING PYTHON WITH SQL

Integrating Python with SQL is one of the most important capabilities in modern data analytics. While SQL is excellent for structured data storage and retrieval, Python provides powerful tools for data cleaning, transformation, visualization, and modeling. By combining both, analysts can build efficient end-to-end data pipelines.

This chapter explains how Python connects with MySQL databases, how queries are executed, how results are transformed into DataFrames using Pandas, and how integration is used in real-world analytics workflows.

7.1 Introduction to Python–SQL Integration

Python–SQL integration enables:

- Automation of SQL queries
- Fetching SQL data directly into Python
- Cleaning and transforming SQL outputs
- Saving processed data back to the database.

7.2 Architecture of Python–SQL Workflow

Steps in Workflow

1. Establish Connection using a connector
2. Send SQL Query to MySQL database
3. Receive Query Output
4. Load Output into Pandas DataFrame
5. Perform Data Cleaning & Analysis

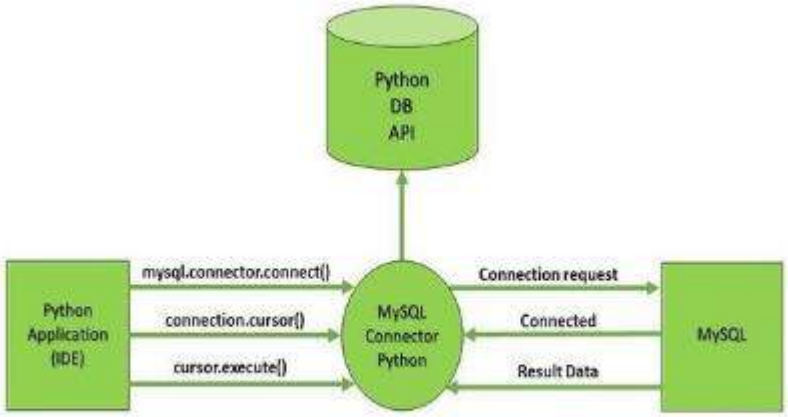


Figure 7.1: Python–SQL Integration Workflow

7.3 Installing SQL Connector in Python

Python requires a connector such as:

- mysql-connector-python
- pymysql
- SQLAlchemy (ORM-based)
- Installation
- pip install mysql-connector-python

7.4 Connecting Python to MySQL

Example Connection Code

```
import mysql.connector

mydb = mysql.connector.connect(

    host="localhost",

    user="root",

    password="12345",

    database="analytics_db"

)
```

```
cursor = mydb.cursor()
```

Explanation

```
host: database server address

user: MySQL username

password: user password

database: name of database
```

7.5 Executing SQL Queries Using Python

SELECT Query

```
cursor.execute("SELECT * FROM employees")

result = cursor.fetchall()

for row in result:

    print(row)
```

INSERT Query

```
query = "INSERT INTO employees (id, name, age, salary) VALUES (%s, %s, %s, %s)"

values = (3, "Karan", 29, 58000)

cursor.execute(query, values)

mydb.commit()
```

commit() saves the transaction.

7.6 Loading SQL Data into Pandas DataFrame

Pandas provides an in-built method read_sql().

Example

```
import pandas as pddf = pd.read_sql("SELECT * FROM employees", mydb)print(df.head())
```

7.7 Data Cleaning After Import

Once loaded into Pandas, various operations can be performed:

```
df.drop_duplicates(inplace=True)
```

```
df['salary'] = df['salary'].fillna(df['salary'].mean())
```

```
df['name'] = df['name'].str.title()
```

These cleaned results can later be exported.

7.8 Exporting Data Back to Database

```
for index, row in df.iterrows():
    cursor.execute(
        "UPDATE employees SET salary=%s WHERE id=%s",
        (row['salary'], row['id'])
    )
mydb.commit()
```

Python makes database updates efficient.

7.9 Real-World Use Case: Sales Dashboard Pipeline

Workflow

- Sales data stored in MySQL
- Python script extracts data daily
- Data cleaned using Pandas
- Exported to a CSV file
- Power BI picks the CSV file
- Dashboard refreshes automatically

This pipeline is used widely in retail, finance, and operations analytics.

CHAPTER 8

INTRODUCTION TO POWER BI

Power BI is a powerful business intelligence and data visualization tool developed by Microsoft. It enables users to connect to multiple data sources, model data, create interactive reports, and build visually appealing dashboards. Power BI is widely used across industries for reporting, analytics, forecasting, and performance monitoring.

8.1 What is Power BI?

Power BI is a Business Intelligence (BI) tool designed to transform raw data into informative insights. It supports:

Data connectivity

Data transformation

Power BI is ideal for analysts, managers, and decision-makers who require real-time reporting.

8.2 PowerBI Desktop Interface

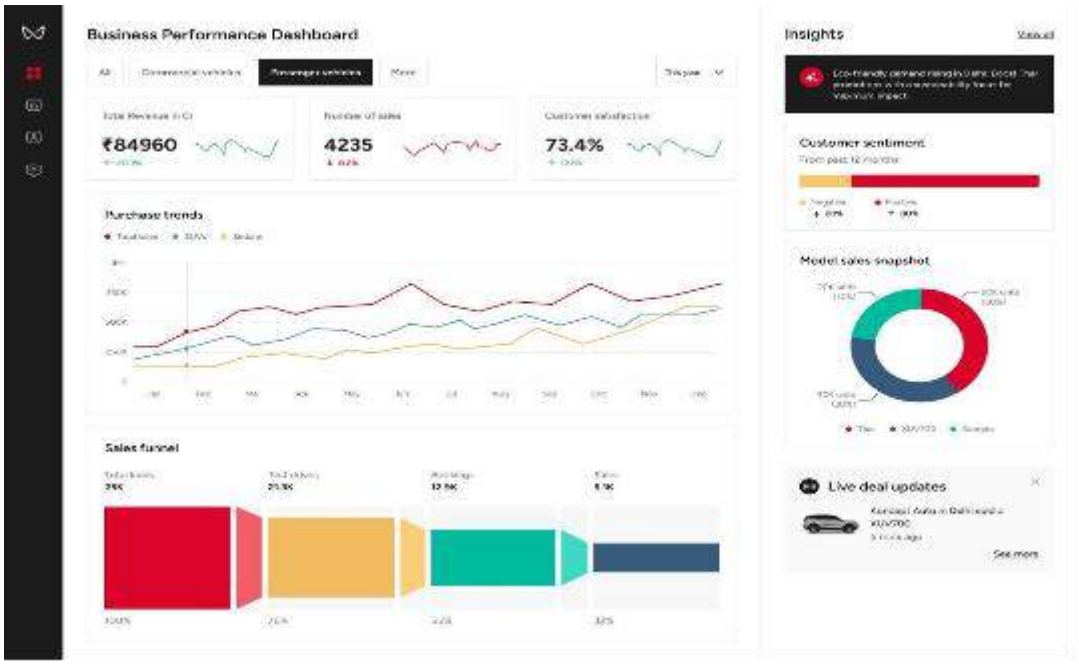


Figure 8.1: Power BI Desktop Interface

The interface consists of:

1. Ribbon Menu

Tools for importing data, visualizations, modeling, and formatting.

2. Report View

Canvas area where visuals and dashboards are designed.

3. Data View

Tabular representation of loaded data.

4. Model View

Shows relationships between tables.

5. Visualization Pane

Contains charts, maps, KPIs, cards, slicers, etc.

6. Fields Pane

Displays tables and columns available for reporting.

8.3 Power Query Editor

Power Query Editor is the engine behind data cleaning and transformation in Power BI.

Features of Power Query Editor

Remove duplicates

Handling missing values

Changing data types

Splitting and merging columns

8.4 Data Modeling in Power BIData modeling is essential for connecting multiple tables and enabling accurate calculations.

Types of Relationships

- One-to-one
- One-to-many
- Many-to-many

8.5 Measures and Calculated Columns (DAX)

DAX (Data Analysis Expressions) is the formula language of Power BI.

Calculated Columns

Computed at row level.

Example:

```
Profit = Sales[Revenue] - Sales[Cost]
```

Measures

Calculated dynamically based on filters.

Popular DAX Functions

- SUM, AVERAGE, MIN, MAX
- CALCULATE
- FILTER
- COUNTROWS

DAX helps create KPIs such as:

8.6 Formatting and Customizing Visuals

Power BI allows customization options such as:

- Colors and themes
- Data labels
- Legends
- Titles and tooltips

Visualizations such as bar charts, KPIs, maps, and slicers are commonly used.

8.7 Advantages of Using Power BI

BenefitExplanation

User-friendly Drag-and-drop interface

Real-time dashboards Supports auto-refresh

Rich visuals Advanced charts and maps

Cloud integration Easy sharing and access

Large community Tutorials and custom visuals

Power BI simplifies the end-to-end analytics workflow for business users.

CHAPTER 9

POWER BI DASHBOARDS & VISUALIZATION

A dashboard is a visual representation of key performance indicators (KPIs), metrics, and insights that allows users to quickly understand business trends and make data-driven decisions. Power BI provides a highly interactive environment for designing dashboards that combine charts, tables, maps, KPIs, and filters.

During the internship, dashboards were built using Power BI Desktop based on datasets related to sales, HR analytics, and operations. This chapter covers the dashboard creation process, visualization techniques, DAX applications, and best practices for designing effective analytical dashboards.

9.1 Introduction to Dashboards

Dashboards display summarized information in a clear and concise way.

Characteristics of an effective dashboard:

- Minimalistic design
- Easy to understand

Dashboards are widely used in industries such as retail, healthcare, finance, HR, marketing, and supply chain management.

9.2 Power BI Dashboard Interface

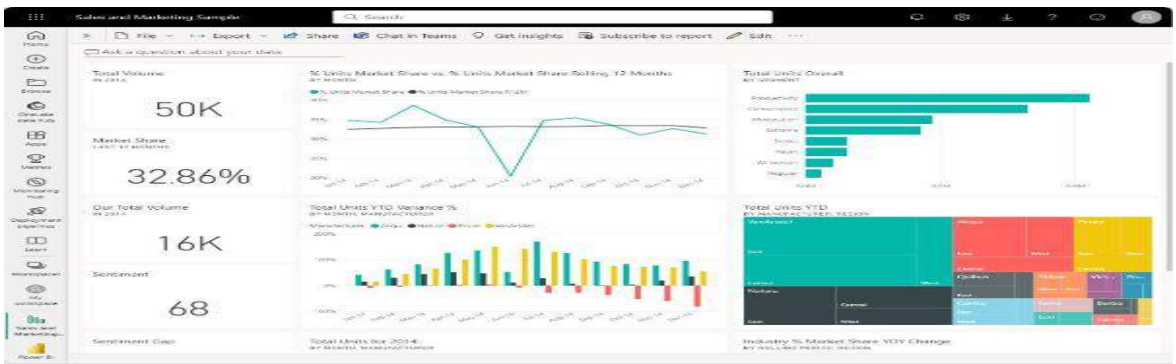


Figure 9.1: Example of a Power BI Dashboard Layout

The dashboard interface allows:

- Drag-and-drop visuals
- Applying filters and slicers
- Adding images, shapes, buttons, and themes
- Custom formatting for charts

9.3 Common Visualization Types and Their Uses

1. Bar/Column Charts

Used to compare categories (e.g., Region vs Sales).

2. Line Charts

Used for trend analysis over time.

3. Pie/Donut Charts

Show proportional contributions (e.g., product category split).

4. Maps

Used for geographical performance.

5. KPI Cards

Show key indicators like:

- Total Sales
- Profit
- YoY Growth

6. Matrix Table

Displays hierarchical or tabular data.

9.4 Creating KPIs in Power BI Using DAX

```
Total Sales

Total Sales = SUM(Sales[Revenue])

Average Monthly Sales

Average Monthly Sales = AVERAGE(Sales[Revenue])

Profit Margin

Profit Margin = DIVIDE(SUM(Sales[Profit]),
SUM(Sales[Revenue]))

Year-over-Year Growth

YoY Growth =

CALCULATE(

    SUM(Sales[Revenue]),

    SAMEPERIODLASTYEAR(Sales[Date])

)
```

KPIs help stakeholders make quick decisions.

9.5 Interactive Filters and Slicers

Slicers allow users to interact with dashboards dynamically.

Common slicers:

- Date slicer
- Region selector
- Product category
- Customer segmentation

9.6 Drill-Through and Tooltips

Drill-Through

Allows navigating to a detailed page by right-clicking on a chart element.

Tooltips

Pop-up info box that displays additional details on hover.

Example: Showing customer count or profit when hovering over a bar.

9.7 Publishing and Sharing Dashboards

To publish:

Click Publish

Choose workspace in Power BI Service

Dashboard becomes available online

Can be shared with:

Team members

Managers

Clients

Automatic data refresh can be scheduled

Importance:

Makes insights accessible

Supports collaborative decision-making

Ensures real-time reporting

9.8 Real-World Use Case: Sales Analytics Dashboard

Problem

A company wants to analyze:

- Revenue trends
- Region-wise sales
- Best-selling products
- Monthly performance

Power BI Dashboard Includes

- KPI Cards: Total Sales, Total Profit, Avg Profit Margin
- Line Chart: Sales over months
- Bar Chart: Sales by region
- Pie Chart: Category-wise distribution
- Map: Country-wise sales
- Slicer: Year, Month, Product Category

Outcome

- Identified most profitable categories
- Highlighted declining regions
- Detected seasonal trends

CHAPTER 10

My Project

1.Objective of the Project :

1. To automate basic library operations such as issuing, returning, and managing books.
2. To maintain an organized digital record of books, students, and transactions.
3. To reduce manual errors and improve the efficiency of library work.
4. To provide a simple, user-friendly interface using Python Tkinter.
5. To store and manage all data securely using a MySQL database.

2. Key Features:

1. **Add New Books**

Allows librarians to enter book title, author, category, and quantity.

2. **Search Books**

Search by book ID, name, or author for quick access.

3. **Issue Books**

Records student ID, book ID, issue date.

4. **Return Books**

Automatically updates stock and calculates late returns (optional).

5. **View All Books / Students**

Shows data in a neat tabular format (Tkinter Treeview).

6. **Update & Delete Records**

Modify book details or remove outdated entries.

7. **MySQL Database**

Proper data storage for long-term usage.

3. System Structure:

3.1 Frontend Layer (Tkinter GUI)

- Windows, labels, buttons, entry fields
- Menus for issuing, returning, searching
- Treeview tables

3.2 Backend Layer (Python Logic)

- Functions for issuing/returning books
- Input validation
- Update availability count

3.3 Database Layer (MySQL)

- Tables: Books, Students, Issue Records
- SQL queries for CRUD operations

4. Implementation Notes:

1. Developed using **Python 3.x**.
2. GUI designed using **Tkinter** widgets (Labels, Buttons, Treeview, Frames).
3. Connected to database using

   import mysql.connector
4. CRUD operations performed using SQL queries.
5. Proper try–except blocks used to handle errors.
6. Separate functions created such as:
    a. add_book()
    b. update_book()
    c. issue_book()
    d. return_book()
    e. display_books()

References

1. Provost, F., & Fawcett, T. (2013). *Data Science for Business*. O'Reilly Media.
2. Davenport, T. H., & Harris, J. G. (2007). *Competing on Analytics: The New Science of Winning*. Harvard Business Review Press.
3. IBM. (n.d.). *What is Data Analytics?* Retrieved from https://www.ibm.com
4. Microsoft. (n.d.). *Introduction to Data Analysis*. Retrieved from https://learn.microsoft.com
5. Kaggle. (n.d.). *Datasets & Data Analysis Tutorials*. Retrieved from https://www.kaggle.com
6. Van Rossum, G., & Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace.
7. Python Software Foundation. (n.d.). *Python Documentation*. Retrieved from https://docs.python.org
8. Matplotlib Developers. (n.d.). *Matplotlib Documentation*. Retrieved from https://matplotlib.org
9. Pandas Team. (n.d.). *Pandas Documentation*. Retrieved from https://pandas.pydata.org
10. Seaborn Developers. (n.d.). *Seaborn Documentation*. Retrieved from https://seaborn.pydata.org
11. DuBois, P. (2013). *MySQL (Developer's Library)*. Addison-Wesley.
12. Oracle. (n.d.). *MySQL Reference Manual*. Retrieved from https://dev.mysql.com/doc
13. Microsoft. (n.d.). *Power BI Documentation*. Retrieved from https://learn.microsoft.com/power-bi
14. Collie, R., & Singh, A. (2017). *Power BI Book*. Holy Macro! Books.